# Mastercard Innovation Challenge 2026

## AI Defence Lab for Payment Security

### *Implemented First-Cut Architecture and Next-Step Options*

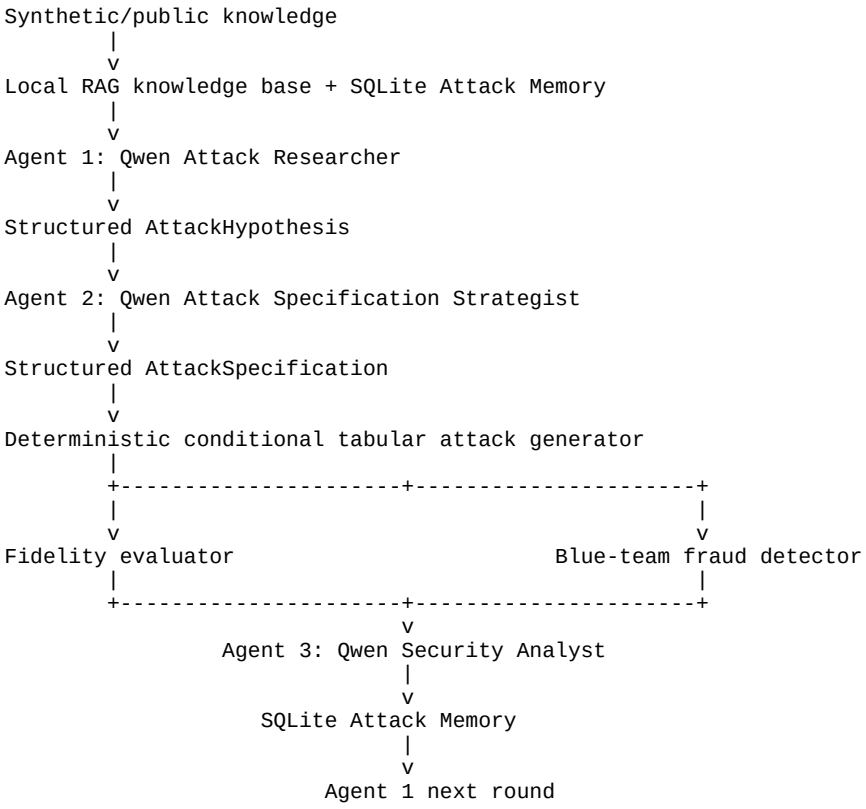
**Status date:** 22 August 2026

## Executive Summary

A first-cut closed-loop red-team/blue-team prototype has been implemented and executed on a Kaggle GPU from VS Code. The system uses one local Qwen2.5-7B-Instruct GGUF Q4_K_M model across three logical roles: Attack Researcher, Attack Specification Strategist, and Security Analyst. The prototype completed two synthetic-data rounds with the required feedback direction: Agent 3 -> Attack Memory -> Agent 1.

The current result is an engineering baseline, not a final competition claim. The detector and loop are working, and the protocol now separates detector-training attacks from unseen evaluation attacks across three configured rounds. Agent 1 also receives a round-specific research query derived from the prior Attack Memory weakness/recommendation. Generator fidelity, attack diversity, novelty, and web-prototype polish still need strengthening before submission.

## Implemented Architecture




The implementation deliberately prevents direct Agent 3 -> Agent 2 feedback. Agent 2 receives the Agent 1 hypothesis and schema constraints only. Agent 3 findings are written to Attack Memory and become available to Agent 1 during the next research round.

## Components Implemented

### *Shared local LLM*

- Model: Qwen2.5-7B-Instruct GGUF, Q4_K_M.
- Current distribution: two GGUF shards totaling approximately 4.361 GiB.
- Storage: local models/ directory and private Kaggle Dataset; model files are excluded from GitHub.
- Runtime: llama-cpp-python CUDA wheel on Kaggle.
- Reuse: one lazily loaded model instance is shared across all three logical roles.
- Structured output: JSON response parsing, bounded output length, retry on malformed JSON, and Pydantic validation.

### *Agent 1: Attack Researcher*

Consumes retrieved evidence, accumulated Attack Memory, and synthetic-scope constraints. Produces an AttackHypothesis with attack family, scenario, target context, behavioural mechanism, novelty rationale, research direction, evidence references, and memory context.

### *Agent 2: Attack Specification Strategist*

Consumes only the Agent 1 hypothesis. Produces an AttackSpecification containing temporal, amount, device, beneficiary, realism, feature, and evasion constraints. It does not receive detector findings directly.

### *Agent 3: Security Analyst*

Consumes detector metrics and fidelity evidence. Produces a WeaknessReport with observed weaknesses, supporting evidence, priority, confidence, and a recommended next attack direction. The report is persisted to Attack Memory.

### *RAG layer*

The first public-research iteration uses a local, curated text knowledge base containing original defensive summaries of reviewed public NIST and ENISA material, plus a project-authored synthetic-data methodology note. Retrieval is a lightweight term-overlap mechanism that returns evidence excerpts and source identifiers. It is intentionally simple and auditable; embedding retrieval remains a later upgrade.

### *Attack Memory*

SQLite stores hypotheses, specifications, evaluation summaries, and weakness reports. The next Agent 1 call retrieves recent records. This separates internal experiment experience from external/public knowledge.

### *Generator*

The first cut is a deterministic conditional tabular baseline, not a GAN or diffusion model. It generates attack rows conditioned on the selected attack family and retains attack ID, family, round, method, and fraud label metadata.

### *Fidelity evaluator*

The evaluator now reports amount mean delta, amount standard-deviation delta, behavioural signal delta, channel coverage, fraud rate, and a heuristic behavioural-plausibility score. It compares unseen generated attacks with a disjoint synthetic reference subset. It is separate from detection so realism and detectability remain distinct objectives.

### *Diversity evidence*

The evaluator now records attack-family count, channel count, unique-row ratio, and numeric-feature variation for each unseen attack batch. These are initial internal indicators, not official Mastercard scoring formulas.

### *Detector*

The blue team uses a scikit-learn pipeline with one-hot channel encoding and a HistGradientBoostingClassifier. It reports precision, recall, F1, ROC-AUC, false-positive rate, and confusion matrix on a holdout-plus-attack evaluation set.

### *Web prototype*

A Streamlit entry point exposes the two-round demonstration and displays hypotheses, specifications, fidelity, detection, and Agent 3 findings.

## Synthetic Data Used

No real cardholder data, PII, production payment data, confidential data, or live-system data was used.

The reference transaction generator creates 400 legitimate synthetic rows per configured run using a fixed seed. Each row contains:

- amount: lognormal synthetic amount.
- hour: integer hour from 0 to 23.
- device_change: binary synthetic signal.
- beneficiary_change: binary synthetic signal.
- velocity_24h: Poisson-distributed short-window transaction count.
- channel: one of web, mobile, or card_present.
- is_fraud: legitimate label 0.

Each round generates 80 synthetic attack rows. The baseline supports attack families including:

- low_and_slow: lower amounts and reduced transaction velocity.
- trusted_device: lower device-change frequency to test detector reliance on device novelty.
- Generic conditional attack rows with higher amount, device-change, beneficiary-change, and velocity distributions.

Generated records retain:

- attack ID,
- attack family,
- round number,
- generation method,
- ground-truth fraud label.

The current dataset is a controlled synthetic demonstration schema. It is not a claim that these distributions represent live Mastercard payment data.

## Validation Evidence

The Kaggle-connected VS Code notebook verified:

- CUDA available: True.

- GPU: Tesla T4.
- GPU count: 2.
- Both Qwen GGUF shards accessible from the private Kaggle Dataset.
- Total model size: approximately 4.361 GiB.
- CUDA-enabled llama.cpp GPU offload support: True.
- Qwen structured JSON inference: passed.
- Three-round Qwen-backed loop: passed for the prior baseline; the updated three-round protocol is locally validated and ready for Kaggle execution.
- Agent backend: QwenAgents.
- Detection F1 in the validated two-round run: 0.988 and 0.988 (historical baseline before the split hardening).
- Feedback path: Agent 3 -> Memory -> Agent 1: OK.

The updated three-round hardened Kaggle run completed with QwenAgents using the reviewed RAG corpus and unseen-attack evaluation. Detection F1 was 0.826, 0.734, and 0.812; fidelity plausibility was 0.4706, 0.5232, and 0.4928. Each round covered three channels and achieved a unique-row ratio of 1.0, while the generated attack batch remained one attack family per round. These values are internal synthetic-experiment evidence, not official Mastercard scores. The latest strict-diversity Qwen Kaggle rerun produced social_engineering, trusted_device, and account_takeover, with novelty scores 0.9446, 0.9517, and 0.9561; detection F1 was 0.826, 0.734, and 0.769; fidelity plausibility was 0.4706, 0.5399, and 0.4793. The novelty score is an internal token-level structured-distance heuristic, not an official Mastercard scoring formula. The controller now enforces distinct abstract families across a three-round run; the seven-family taxonomy is account_takeover, trusted_device, beneficiary_manipulation, low_and_slow, social_engineering, merchant_abuse, and cross_channel_anomaly.

The historical F1 values are results from the earlier synthetic baseline and should not be presented as official competition scores or real-world deployment performance. The hardened protocol should report new unseen-evaluation results before drawing comparisons.

## Rules and Compliance Position

The supplied Rules snapshot requires a code repository, solution walkthrough, and working web prototype. It permits synthetic, anonymized, or authorized sample data and prohibits real cardholder data, PII, production payment data, live-system targeting, payment-infrastructure targeting, and third-party targeting.

The current implementation follows these boundaries:

- Synthetic-only data in iteration 1.
- Offline attack generation and evaluation.
- No live payment-system interaction.
- Model weights kept outside GitHub in a private Kaggle Dataset.
- Public research sources for iteration 2 must be reviewed for authorization, licensing, confidentiality, and provenance before ingestion.
- The Streamlit prototype and GitHub repository are present as implementation artifacts.

## Logical Next-Step Options

### *Option A: Submission-hardening baseline*

Recommended first. Keep the deterministic generator, but improve evaluation and reproducibility:

1. Add explicit train, generator-fit, detector-train, validation, and unseen-test splits.
2. Add per-attack-family precision, recall, F1, PR-AUC, ROC-AUC, calibration, and false-positive analysis.
3. Add attack diversity, deduplication, novelty, and round-over-round reports.
4. Save every configuration, prompt, model identifier, seed, and output artifact.
5. Make the Streamlit demo show the two-round feedback change clearly.

**Benefit:** highest confidence and lowest compute risk before submission.

### *Option B: Public-research RAG upgrade*

Recommended for iteration 2. Add a small, reviewed corpus of public fraud research, security reports, typologies, and permitted documentation.

1. Record source URL, title, publisher, date, license, and retrieval date.
2. Chunk documents with stable source IDs.
3. Replace term overlap with embedding retrieval and optional reranking.
4. Require evidence references in Agent 1 output.
5. Add a source-review manifest so no confidential or unauthorized material enters the corpus.

**Benefit:** stronger identification breadth, evidence grounding, and novelty rationale.

### *Option C: Stronger tabular generator*

Use only after Option A is stable and only in Kaggle GPU sessions:

1. Establish a baseline against the deterministic generator.
2. Try a conditional tabular model such as CTGAN or a tabular diffusion implementation.
3. Fit only on permitted synthetic/reference data.
4. Keep the final detector evaluation set unseen by generator fitting.
5. Compare marginal distributions, dependencies, conditional behaviour, downstream detector utility, and privacy/leakage sanity checks.

**Benefit:** improved fidelity and diversity; higher runtime, debugging, and evaluation risk.

### *Option D: Detector robustness upgrade*

Add stronger features or models only if baseline evidence justifies it:

- threshold tuning for false-positive control,
- calibrated probabilities,
- temporal/velocity aggregates,
- sequence or graph features if the schema expands,
- model comparison with XGBoost or LightGBM.

**Benefit:** stronger defend pillar; risk of overfitting synthetic artifacts if evaluation splits are weak.

## Recommended Order

1. Freeze and document the validated Qwen baseline with the hardened three-round protocol.

2. Run the hardened protocol on Kaggle and save the new unseen-evaluation results.

3. Upgrade RAG with embedding retrieval over the seven-document reviewed public-summary corpus while retaining the allowlist and source manifest.

4. Run the new strict family-diversity policy on Kaggle and compare its unseen-evaluation results with the prior Qwen run.

5. Compare a stronger conditional generator against the baseline on Kaggle GPU.

6. Improve the Streamlit walkthrough and package the code repository plus solution document.

7. Run a final compliance, provenance, privacy, and reproducibility audit before submission.

## Current Limitations

- The generator is deterministic and simple; it is not yet CTGAN, diffusion, or adversarial training.
- The RAG corpus is a small reviewed public-summary collection, not a broad research collection.
- Fidelity and diversity scores are internal heuristics and not official Mastercard scoring formulas.
- The current synthetic distributions are not evidence of live-payment behaviour.
- The validated two-round run demonstrates engineering closure, not production readiness.
- More comprehensive unseen evaluation and attack novelty analysis are required before making strong competition claims.

## Final Position

The project has a working, competition-aligned first cut: Identify through reviewed RAG and memory, Generate through structured specifications and synthetic tabular generation, and Defend through a detector evaluated on unseen attacks. The most defensible next move is to run and archive the hardened Kaggle results, then increase public-research breadth and generator realism.

## Questions and Direct Answers

### *1. Does Agent 1 actually receive public or external knowledge?*

Not in a meaningful research-grade sense yet. Agent 1 does receive the output of the RAG component, but the current knowledge directory contains only one short, locally authored synthetic seed document. That document mentions generic fraud concepts such as trusted-device abuse, beneficiary manipulation, and low-and-slow behaviour. It is not a public research corpus, and it is not evidence of Mastercard or live-payment knowledge.

This does not violate the rules because it uses no restricted data. However, it means the current Identify pillar is only a plumbing demonstration. Iteration 2 should add reviewed public sources with provenance and licensing metadata, then retrieve those sources for Agent 1.

### *2. How does Agent 3 consume fidelity evidence?*

The controller computes a fidelity dictionary after generation and passes that dictionary, together with detector metrics, to Agent 3. Agent 3 does not independently calculate fidelity. It reasons over the evaluator output.

The current fidelity evidence is limited to amount mean difference, amount standard-deviation difference, fraud rate, and a heuristic plausibility score. It is not a full real-world realism assessment. Agent 3 can therefore discuss the current evaluator evidence, but its conclusion is only as strong as that evaluator.

### *3. How does the RAG layer work if no public knowledge base was supplied?*

It works technically, but only as a minimal local retrieval demonstration. The local text file is loaded, a query is split into terms, documents are scored by term overlap, and the top matching excerpt plus source metadata is passed to Agent 1. Because there is only one seed document, retrieval is not yet competitive RAG.

The correct upgrade is a reviewed corpus of public fraud research, security reports, typologies, and permitted documentation. Each document should carry source URL, title, publisher, date, license or usage basis, and retrieval date.

### *4. What is the synthetic seed document?*

It is data/knowledge_base/seed_fraud_typologies.txt, a short planning aid written for the local demonstration. It says that payment fraud can involve social engineering, account takeover, trusted-device abuse, beneficiary manipulation, and low-and-slow patterns, and that amount, time, device, beneficiary, channel, and velocity signals can matter.

It is not scraped research, not confidential information, not a production dataset, and not a substitute for the public knowledge base required for a stronger Identify pillar.

### *5. How can the deterministic generator generate attacks without a base knowledge source?*

It does not learn a real distribution. It uses manually chosen synthetic probability distributions in code. Legitimate reference rows use lognormal amounts, random hours, low probabilities of device and beneficiary changes, Poisson velocity, and channel probabilities. Attack rows use altered distributions; the low_and_slow family lowers amount and velocity, while trusted_device lowers device-change frequency.

The Agent 2 specification selects the family and carries metadata, but the baseline generator does not yet translate every specification field into a learned conditional distribution. This is a transparent smoke-test generator, not a high-fidelity fraud simulator.

### *6. How can the fidelity evaluator evaluate without public base knowledge?*

It compares generated attack rows with the synthetic legitimate reference distribution. It calculates amount mean delta, amount standard-deviation delta, fraud rate, and a heuristic plausibility score. Therefore it evaluates consistency with the local synthetic schema, not similarity to public or real payment data.

The current result should be described as synthetic-schema consistency. A stronger evaluator must compare permitted reference data across marginals, correlations, conditional relationships, temporal behaviour, downstream utility, privacy leakage, and attack-specific constraints.

### *7. Why were all competition criteria not tracked?*

The rules state that the judging panel may consider diversity, fidelity, detector efficacy, novelty, real-world feasibility, innovation, originality, technical quality, scalability, commercial viability, and presentation quality. They do not provide a required public formula or mandatory implementation for every criterion.

The first cut tracked only part of detector efficacy and a small fidelity proxy because the immediate milestone was a closed loop. That is incomplete for a competition submission. The next evaluation layer should add:

- **Diversity:** attack-family coverage, mechanism count, behavioural-feature diversity, and deduplication.

- **Fidelity:** marginal and dependency similarity, conditional plausibility, constraint violations, downstream utility, and privacy/leakage checks.
- **Detection efficacy:** precision, recall, F1, ROC-AUC, PR-AUC, calibration, confusion matrix, threshold trade-offs, and false-positive rate.
- **Novelty:** distance from prior specifications, mechanism novelty versus parameter changes, and newly exposed detector weaknesses.
- **Real-world feasibility:** latency, throughput, resource requirements, offline deployment shape, auditability, robustness under distribution shift, and why generation is outside the live-payment critical path.
- **Innovation, originality, scalability, commercial viability, and presentation:** documented design rationale, ablations, reproducible artifacts, deployment architecture, and a polished Streamlit walkthrough.

These should be presented as internal evidence aligned to the rules, not as claims about Mastercard's secret judging weights.

### *8. What inspired the synthetic data generation?*

The inspiration came from the architecture brief's generic payment-security feature examples and the need for a safe, fully synthetic schema. The selected signals represent common abstract transaction dimensions: amount, time, channel, device change, beneficiary change, and short-window velocity.

They were not learned from a Mastercard dataset and do not claim to represent live payment behaviour. Public research in iteration 2 should be used to justify broader attack families and more realistic relationships.

### *9. What is each round of synthetic generation?*

One round is one complete closed-loop experiment:

1. Agent 1 retrieves seed/public evidence and prior Attack Memory.
2. Agent 1 proposes an attack hypothesis.
3. Agent 2 converts that hypothesis into a specification.
4. The generator creates 80 synthetic attack rows for the round.
5. The fidelity evaluator scores the generated rows.
6. The detector is trained/evaluated.
7. Agent 3 analyzes detector and fidelity evidence.
8. The hypothesis, specification, evaluation, and weakness report are stored in Attack Memory.
9. The next round gives Agent 1 access to the accumulated memory.

The current configuration creates 400 legitimate reference rows and 80 generated attack rows per round, with two rounds in the validated demonstration.

### *10. Why are there only a few attributes?*

The small schema was chosen for a deterministic first-cut smoke test and easy auditing, not because other attributes are unimportant. Important future dimensions could include merchant category, geography, currency, authentication method, card-present indicators, account age, beneficiary age, device reputation, IP or network risk, session duration, transaction sequence, failed attempts, cross-border indicators, and graph relationships.

Adding attributes without permitted evidence and a clear evaluation design would only create synthetic complexity, not realism. The next schema should be justified by reviewed public sources or authorized sample data, and each new feature should have a defined generation rule, detector use, fidelity test, and privacy review.

### *11. What is the two-round Qwen loop, and what is Agent Backend?*

The two-round Qwen loop is the validated Kaggle demonstration in which the same Qwen model is invoked with three different prompts: Agent 1 Researcher, Agent 2 Strategist, and Agent 3 Security Analyst. Round 2 receives the weakness records written by Agent 3 during round 1 through SQLite Attack Memory.

Agent backend: QwenAgents means the controller selected the Qwen-backed implementation rather than the deterministic HeuristicAgents fallback. It does not mean three models were loaded. One shared Qwen model instance was reused for the three logical roles.

### *12. Why was F1 so high?*

The high F1 is not evidence that the detector is strong in a realistic setting. It is mainly explained by an easy synthetic problem and an evaluation leakage risk. The attack rows were appended to the detector training data and then the same generated attack batch was included in evaluation. The attack distributions also deliberately differ from legitimate rows in highly informative features such as amount, device change, beneficiary change, and velocity.

Consequently, the reported 0.988 F1 is a smoke-test result for pipeline execution, not a trustworthy generalization estimate. The detector must be retrained and evaluated with strict unseen splits: attacks unseen during detector training, later attack families, held-out parameter ranges, and a legitimate holdout. The report should then include per-family results, false positives, threshold analysis, and distribution-shift performance.

## Interpretation After This Q&A

The first cut proves orchestration, model integration, structured contracts, synthetic generation, detector execution, and the required Agent 3 -> Attack Memory -> Agent 1 feedback direction. It does not yet prove public-research coverage, realistic fidelity, attack novelty, robust detector generalization, or production feasibility.

The highest-value next step is evaluation hardening: introduce clean unseen splits and implement diversity, fidelity, novelty, and feasibility reports before upgrading the generator. This prevents a more complex generator from hiding weaknesses that the current easy synthetic setup exposes.